

MUHAMMAD EHSANUL KADER

✉ mukader@cs.stonybrook.edu

🌐 ehsanulkader.github.io

🌐 ehsanulkader

🌐 ehsanulkader

WORK EXPERIENCE

Lecturer, CSE Department, Brac University

July 2024 – July 2025

Courses Taught : Operating Systems Algorithms Data Structures Introductory Programming Language Lab

EDUCATION

PhD in Computer Science

August 2025 – Present

[Stony Brook University](#)

Courses Completed : Distributed Systems Machine Learning Ongoing Courses : Database Systems Data Science Logic in CS

Bachelor of Science in Computer Science & Engineering

April 2019 – June 2024

[Bangladesh University of Engineering & Technology](#)

CGPA : 3.79/4.0

RESEARCH AND PUBLICATIONS

Dependency, Deadline and Priority Aware Multi-Queue Dynamic Task Scheduling Using Heterogeneous Resources in Fog Environment Task Scheduling Fog Computing 2023 - 2024

- Proposed an algorithm for task scheduling in fog systems, addressing priority levels, deadline constraints, and task dependencies utilizing Directed Acyclic Graphs (DAGs), priority queues, and dynamic queue switching.
- Contributed to algorithm development by identifying limitations and making strategic adjustments to enhance performance.
- Implemented simulation programs in Java and recommended configurations for experiments.
- Simulation results showed significant improvements in response time, makespan, throughput, and task completion rate, particularly with tasks that have inter-dependencies.

Published in *IEEE/ACM 17th International Conference on Utility and Cloud Computing* [\[DOI\]](#)

MD-CardioNet: A Multi-Dimensional Deep Neural Network for Cardiovascular Disease Diagnosis from Electrocardiogram Multidimensional CNN Knowledge Distillation 2021 - 2023

- Developed an efficient deep learning architecture with sequential 1D, 2D, and 3D feature extractors to capture intra- and inter-channel dependencies in ECG signals, improving cardiovascular disease detection accuracy.
- Introduced a novel knowledge distillation framework that transfers knowledge from a high-performing teacher model to a student model with significantly fewer parameters.
- Achieved satisfactory performance in the student model while maintaining efficiency and reducing model complexity.

Published in *IEEE Journal of Biomedical and Health Informatics* [\[DOI\]](#)

Forecasting COVID-19 cases: A comparative analysis between recurrent and convolutional neural networks Neural Networks Time Series analysis 2020 - 2021

- Compared the forecasting capabilities of several deep learning models, including CNN, LSTM, GRU, and multivariate CNN, for predicting COVID-19 cases in three countries.
- While RNN models are generally recognized for their effectiveness with time series data, in our experiments CNNs outperformed them by effectively learning local data patterns, as indicated by performance metrics such as MSE, nRMSE, and MAPE.

Published in *Results in Physics* [\[DOI\]](#)

NOTABLE ACADEMIC PROJECTS

Fault-Tolerant Sharded Distributed Banking System 2PC Multi Paxos Go gRPC

- Developed a fault-tolerant distributed transaction processing system serving 9,000 users using replicated state machine architecture, with a base configuration of 3 clusters and 9 nodes.
- Implemented dynamic cluster reconfiguration allowing changes from the base configuration to different numbers of clusters and nodes per cluster, with users evenly distributed across shards.

- Built a Stable-Leader Multi-Paxos protocol ($2f+1$ quorum model) to provide crash-fault-tolerant consensus within each shard, and Two-Phase Commit (2PC) protocol on top to support cross-shard atomic transactions across clusters.
- Developed lock management and Write-Ahead Logging (WAL) to support abort handling and failure recovery.
- Built benchmarking framework with configurable read/write ratio, cross-shard ratio, and workload skew; measured throughput and latency.

Byzantine Fault-Tolerant Banking System

Linear PBFT

Go

gRPC

- Implemented a Linear-PBFT consensus protocol supporting 7 replicas ($f = 2$ Byzantine faults), reducing communication complexity from $O(n^2)$ to $O(n)$ via collector-based aggregation.
- Developed the full view-change protocol under partial synchrony, incorporating pre-prepare, prepare, commit, and new-view reconstruction.
- Implemented checkpointing to enable garbage collection and efficient state recovery across views.
- Integrated RSA-based public/private key cryptography for authenticated messaging and secure quorum validation.
- Simulated Byzantine failures including equivocation, invalid signatures, crash, in-dark, and timing attacks to evaluate safety and liveness.

SyncInc

Django

DjangoREST

ReactJS

MaterialUI

PostgreSQL

[Github](#)

- Developed SyncInc as part of a three person group, a web-based software that enables organizations to efficiently manage and track projects and tasks for enhanced collaboration.
- Used Django and Django REST Framework for backend and API development, ReactJS with Material-UI for frontend, PostgreSQL for database management.
- Used Docker to run the backend Django application in a containerized environment, resolving dependency installation issues.
- Deployed the frontend on Vercel and hosted the backend and database separately on Render, with Firebase Storage used for file storage.

Music Instruments Classification

Python

PyTorch

Librosa

[Github](#)

- Developed a system to identify musical instruments in audio recordings.
- Processed raw audio with Librosa to generate Mel Spectrograms for feature extraction.
- Used a ResNet-18 architecture with additional layers to classify 20 different instruments.

Feature Implementation on OS

xv6

C

[Github](#)

- Added new features to xv6, including system calls for `trace` and `sysinfo`.
- Implemented a lottery scheduling algorithm for probabilistic time slice allocation.
- Enhanced memory management with Copy on Write (COW) and paging.

Implementation of AQMRD Algorithm in NS2

RED

NS2

[Github](#)

- Implemented Adaptive Queue Management with Random Dropping (AQMRD) algorithm that reduces the limitations of Random early detection (RED) active queue management (AQM) scheme.
- Added an adaptive threshold between the lower and upper limits that incorporates both the average queue size and its rate of change

TECHNICAL SKILLS

- **Programming Languages:** C/C++, Java, Python, Javascript, SQL, Bash
- **Tools & Softwares:** Autopsy, Wireshark, NS-2, CloudSim Plus, Docker
- **Frameworks & Libraries:** Django, DjangoREST, ReactJS, Material UI, PyTorch, Tensorflow, Sklearn, Pandas, Matplotlib
- **Database & ORM:** Oracle, PostgreSQL, Django ORM

HONORS & ACHIEVEMENTS

Dean's List

Recipient of Dean's List Scholarship for the year January 2019, July 2021 for academic excellence.

10th Position in BUET CSE FEST 2022 AI Competition

Built an AI that will fight with other AI's to win a game hosted in codingame platform